Seeneyu API Reference

> *Prepared for: Investor Tech Audit Workshop Date: 2026-03-29 | Version: 1.0 Total Endpoints: 132 route files | Framework: Next.js 14 Route Handlers*

---

1. API Overview

Metric	Value
Total Route Files	132
Public Endpoints	~25
Authenticated Endpoints	~40
Admin Endpoints	~65
Cron Endpoints	2
Auth Strategy	NextAuth JWT (web) + Bearer Token (mobile)
Response Format	JSON
Error Format	`{ error: string }` with appropriate HTTP status

---

2. Authentication

Web (NextAuth.js v4)

```
// Session strategy: JWT
// Provider: Credentials (email + password)
// Password hashing: bcryptjs
// Session token: HTTP-only cookie

// Auth check pattern in route handlers:
import { getServerSession } from 'next-auth'
import { authOptions } from '@/lib/auth'

const session = await getServerSession(authOptions)
if (!session) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
```

Mobile (JWT Bearer)

```
// Endpoint: POST /api/mobile/login
// Returns: { token: string, user: {...} }
// Usage: Authorization: Bearer <token>
// Token library: jsonwebtoken
```

Role-Based Access

Role	Access Level	Check Pattern
Public	No auth required	No session check
User	Authenticated learner	`session.user` exists
Admin	`role === 'admin'`	`session.user.role === 'admin'`
Cron	Vercel Cron header	`x-vercel-cron` header or internal call

Approval Gate

New users register with `status: 'pending'`. Only `status: 'approved'` users can sign in. Admin must approve via `/api/admin/users/[id]`.

---

3. Public Endpoints (No Auth Required)

Content & CMS

Method	Path	Description
GET	`/api/clips`	List all active clips with filters
GET	`/api/clips/[id]`	Clip detail with annotations, practice steps
GET	`/api/foundation`	List foundation courses
GET	`/api/foundation/[courseSlug]`	Course detail with lessons
GET	`/api/foundation/[courseSlug]/[lessonSlug]`	Lesson content + examples + quiz
GET	`/api/arcade`	List arcade data
GET	`/api/arcade/bundles`	Arcade bundle list
GET	`/api/arcade/bundles/[id]/challenges`	Bundle challenges
GET	`/api/public/plans`	Subscription plan list for pricing page
GET	`/api/cms/blog`	Published blog post list
GET	`/api/cms/blog/[slug]`	Blog post detail
GET	`/api/cms/pages/[slug]`	CMS page content
GET	`/api/cms/settings/[key]`	Site setting value

Mini-Games (Public, No Auth)

Method	Path	Description
GET	`/api/public/games`	List active mini-games
GET	`/api/public/games/[type]`	Game rounds for a specific game
POST	`/api/public/games/[type]/submit`	Submit answer for a round

POST	`/api/public/games/[type]/complete`	Complete game session, record score
POST	`/api/public/games/[type]/capture`	Submit expression camera capture
GET	`/api/public/games/leaderboard/[type]`	Game-specific leaderboard
GET	`/api/public/certificate/[sessionId]`	Generate/serve certificate image

Error Logging

Method	Path	Description
POST	`/api/logs`	Client-side error reporting (rate-limited)

Request body:

```
{
  "level": "error",
  "message": "Unhandled exception in MediaPipe",
  "stack": "Error: ...",
  "metadata": { "page": "/arcade", "browser": "Chrome 124" }
}
```

---

4. Authenticated Endpoints (User Required)

Auth & Profile

Method	Path	Description
POST	`/api/auth/signup`	Register new account (returns `pending` status)
GET/POST	`/api/auth/[...nextauth]`	NextAuth.js handlers (signin, signout, session)
POST	`/api/mobile/login`	Mobile JWT login
GET	`/api/user/me`	Current user info (mobile)
GET	`/api/user/profile`	User profile data
POST	`/api/onboarding/complete`	Complete 5-skill onboarding assessment

Signup request:

```
{
  "email": "learner@example.com",
  "password": "securePassword123",
  "name": "Jane Doe"
}
```

Signup response:

```
{
  "status": "pending",
  "message": "Account created. Awaiting admin approval."
}
```

Recording & Feedback

Method	Path	Description
POST	`/api/sessions`	Create full performance recording session
POST	`/api/sessions/[id]/feedback`	Request AI feedback (GPT-4o or MediaPipe)
POST	`/api/micro-sessions`	Create micro-practice session

Session creation request:

```
{
  "clipId": "clx1234...",
  "recordingUrl": "https://blob.vercel-storage.com/...",
  "frameUrls": "[\"url1\", \"url2\", ...]"
}
```

Foundation Learning

Method	Path	Description
GET	`/api/foundation/progress`	All course progress for current user
POST	`/api/foundation/progress`	Record lesson completion
GET	`/api/foundation/progress/[lessonId]`	Specific lesson progress
POST	`/api/foundation/progress/[lessonId]`	Submit quiz score

Arcade

Method	Path	Description
POST	`/api/arcade/attempts`	Submit arcade challenge attempt

Attempt request:

```
{
  "challengeId": "clx5678...",
  "score": 85,
  "breakdown": {
    "expression_accuracy": 90,
    "timing": 80,
    "confidence": 85
```

```
  }
}
```

Comments / Discussions

Method	Path	Description
GET	`/api/comments`	List comments (filtered by lessonId or challengeId)
POST	`/api/comments`	Create comment or reply
PATCH	`/api/comments/[id]`	Edit own comment
DELETE	`/api/comments/[id]`	Delete own comment

AI Assistant (Coach Ney)

Method	Path	Description
POST	`/api/assistant/chat`	Send message to Coach Ney (streaming response)
GET	`/api/assistant/conversations`	List conversation history

Chat request:

```
{
  "message": "How can I improve my eye contact?",
  "conversationId": "clx_conv123",
  "context": "foundation_lesson",
  "voice": true
}
```

Gamification

Method	Path	Description
GET	`/api/gamification/profile`	XP, level, streak, hearts
POST	`/api/gamification/activity`	Record gamification activity (triggers XP + streak + quests + badges)
GET	`/api/gamification/badges`	User's earned badges
GET	`/api/gamification/quests`	Today's daily quests
GET	`/api/gamification/leaderboard`	Global weekly leaderboard
GET	`/api/gamification/tier`	User subscription tier info
POST	`/api/gamification/follow`	Follow/unfollow another user
GET	`/api/gamification/feed`	Activity feed (following)

Subscriptions & Payments

Method	Path	Description
GET	`/api/subscriptions`	Current user's subscription
POST	`/api/payments/paypal`	Create PayPal checkout order
POST	`/api/payments/vnpay`	Create VNPay payment URL
GET	`/api/payments/vnpay/callback`	VNPay redirect callback

Other

Method	Path	Description
POST	`/api/activity/track`	Track user activity event
GET	`/api/dashboard/tracks`	Personalized learning path tracks
GET	`/api/submissions`	User's practice submissions

---

5. Admin Endpoints (`/api/admin/*`)

All admin endpoints require `session.user.role === 'admin'` .

Clip Management

Method	Path	Description
GET	`/api/admin/clips`	List all clips (with pagination)
POST	`/api/admin/clips`	Create new clip
GET	`/api/admin/clips/[id]`	Clip detail
PATCH	`/api/admin/clips/[id]`	Update clip
DELETE	`/api/admin/clips/[id]`	Delete clip
POST	`/api/admin/clips/[id]/observation`	Generate observation guide (GPT-4o)
POST	`/api/admin/clips/[id]/crawl-screenplay`	Crawl screenplay from web
POST	`/api/admin/clips/[id]/transcript`	Fetch YouTube transcript
POST	`/api/admin/clips/[id]/screenplay`	Save screenplay text

User Management

Method	Path	Description
GET	`/api/admin/users`	List users with search + status filter
POST	`/api/admin/users`	Bulk user operations
PATCH	`/api/admin/users/[id]`	Update user (approve/reject/suspend)

GET	`/api/admin/users/[id]/profile`	Detailed user profile
GET	`/api/admin/users/[id]/history`	User activity history
GET	`/api/admin/users/[id]/subscription`	User subscription details

Arcade Management

Method	Path	Description
GET/POST	`/api/admin/arcade/bundles`	List/create bundles
GET/PATCH/DELETE	`/api/admin/arcade/bundles/[id]`	Bundle CRUD
GET/POST	`/api/admin/arcade/challenges`	List/create challenges
PATCH/DELETE	`/api/admin/arcade/challenges/[id]`	Challenge CRUD
PATCH	`/api/admin/arcade/challenges/[id]/reorder`	Reorder challenge
POST	`/api/admin/arcade/upload-image`	Upload reference image to Blob

Analytics

Method	Path	Description
GET	`/api/admin/analytics`	Dashboard: DAU/WAU/MAU, signups, sessions
GET	`/api/admin/analytics/features`	Feature usage metrics
GET	`/api/admin/analytics/users/[id]`	Per-user activity detail

Content Discovery (Crawler)

Method	Path	Description
GET/POST	`/api/admin/crawl-jobs`	List/create crawl jobs
GET/PATCH	`/api/admin/crawl-jobs/[id]`	Job detail/update
POST	`/api/admin/crawl-jobs/[id]/run`	Execute crawl job (YouTube API + GPT-4o scoring)
PATCH	`/api/admin/crawl-jobs/[id]/results/[resultId]`	Approve/reject crawl result

CMS

Method	Path	Description
GET/POST	`/api/admin/cms/pages`	CMS page list/create
GET/PATCH/DELETE	`/api/admin/cms/pages/[slug]`	CMS page CRUD
GET/POST	`/api/admin/cms/blog`	Blog post list/create

GET/PATCH/DELETE	`/api/admin/cms/blog/[slug]`	Blog post CRUD
GET/POST	`/api/admin/cms/team`	Team member list/create
PATCH/DELETE	`/api/admin/cms/team/[id]`	Team member CRUD
GET/PUT	`/api/admin/cms/settings/[key]`	Site settings read/write
POST	`/api/admin/cms/upload`	Upload image to Blob

Subscription Plans

Method	Path	Description
GET/POST	`/api/admin/plans`	Plan list/create

Error Logs

Method	Path	Description
GET	`/api/admin/logs`	Error log list with filters
PATCH	`/api/admin/logs/[id]`	Mark log as resolved
DELETE	`/api/admin/logs/[id]`	Delete log entry

Comment Moderation

Method	Path	Description
GET/POST	`/api/admin/comments`	Comment list/moderation
PATCH	`/api/admin/comments/[id]`	Hide/unhide comment

Toolkit

Method	Path	Description
GET/POST	`/api/admin/toolkit/crawler/jobs`	Crawler job CRUD
GET/PATCH	`/api/admin/toolkit/crawler/jobs/[id]`	Job operations
POST	`/api/admin/toolkit/crawler/enrich`	Content enrichment
GET/POST	`/api/admin/toolkit/crawler/expressions`	Expression asset management
PATCH	`/api/admin/toolkit/crawler/expressions/[id]`	Expression operations
GET/POST	`/api/admin/toolkit/mini-games`	Mini-game CRUD
GET/PATCH	`/api/admin/toolkit/mini-games/[id]`	Game operations
POST	`/api/admin/toolkit/mini-games/[id]/rounds`	Add game rounds
PATCH	`/api/admin/toolkit/mini-games/rounds/[roundId]`	Edit round

GET	`/api/admin/toolkit/mini-games/sessions`	Game session data
GET	`/api/admin/toolkit/mini-games/submissions`	Expression submissions
PATCH	`/api/admin/toolkit/mini-games/submissions/[id]`	Approve/reject submission
GET	`/api/admin/toolkit/mini-games/analytics`	Game analytics

Data Pipeline

Method	Path	Description
GET/POST	`/api/admin/data/exports`	Data export list/create
GET	`/api/admin/data/exports/[id]`	Export detail/download
GET/POST	`/api/admin/data/labels`	Training data labels
GET	`/api/admin/data/stats`	Data pipeline statistics
GET	`/api/admin/data/submissions`	Raw submissions for labeling

Import

Method	Path	Description
POST	`/api/admin/import-zip`	Import clips from ZIP archive

---

6. Cron Endpoints

Path	Schedule	Purpose
`/api/cron/morning`	`0 8 * * *`	Morning cycle: analyze users, generate plans, schedule notifications
`/api/cron/engagement-check`	`0 22 * * *`	Evening cycle: streak warnings, process notification queue

---

7. Rate Limiting Strategy

Endpoint Category	Strategy	Limit
Public error logging	IP-based	Implemented per-request check
AI endpoints (chat, feedback)	`maxDuration` config	Extended timeout for OpenAI calls
Mini-game submissions	No limit (public)	Consider adding at scale
Auth endpoints	NextAuth built-in	Standard session management

Current state: Rate limiting is lightweight. At scale (10K+ users), the following should be added:

- Redis-based rate limiting middleware

- Per-user API quotas tied to subscription plan
- IP-based throttling on public endpoints

---

8. CORS Configuration

CORS is handled at the Vercel level with default Next.js settings:

- Same-origin requests: unrestricted
- Cross-origin: allowed for embed routes (mini-games use PostMessage bridge)
- Mobile app: uses direct API calls with Bearer token (no CORS issues for native)

---

9. Error Handling Pattern

All API routes follow a consistent error handling pattern:

```
export async function POST(request: Request) {
  try {
    const session = await getServerSession(authOptions)
    if (!session) {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
    }

    const body = await request.json()
    // Zod validation where applicable
    const parsed = schema.safeParse(body)
    if (!parsed.success) {
      return NextResponse.json({ error: parsed.error.message }, { status: 400 })
    }

    // Business logic...
    return NextResponse.json(result)
  } catch (error) {
    console.error('API error:', error)
    return NextResponse.json(
      { error: 'Internal server error' },
      { status: 500 }
    )
  }
}
```

Document prepared by Data Engineer role. Endpoint count verified against `src/app/api/` directory (132 route files) as of 2026-03-29.